



Lecture 12: Programmatic Component Instantiation

INTERVIEW QUESTION

How do you instantiate a compiled Svelte component programmatically within vanilla JavaScript?

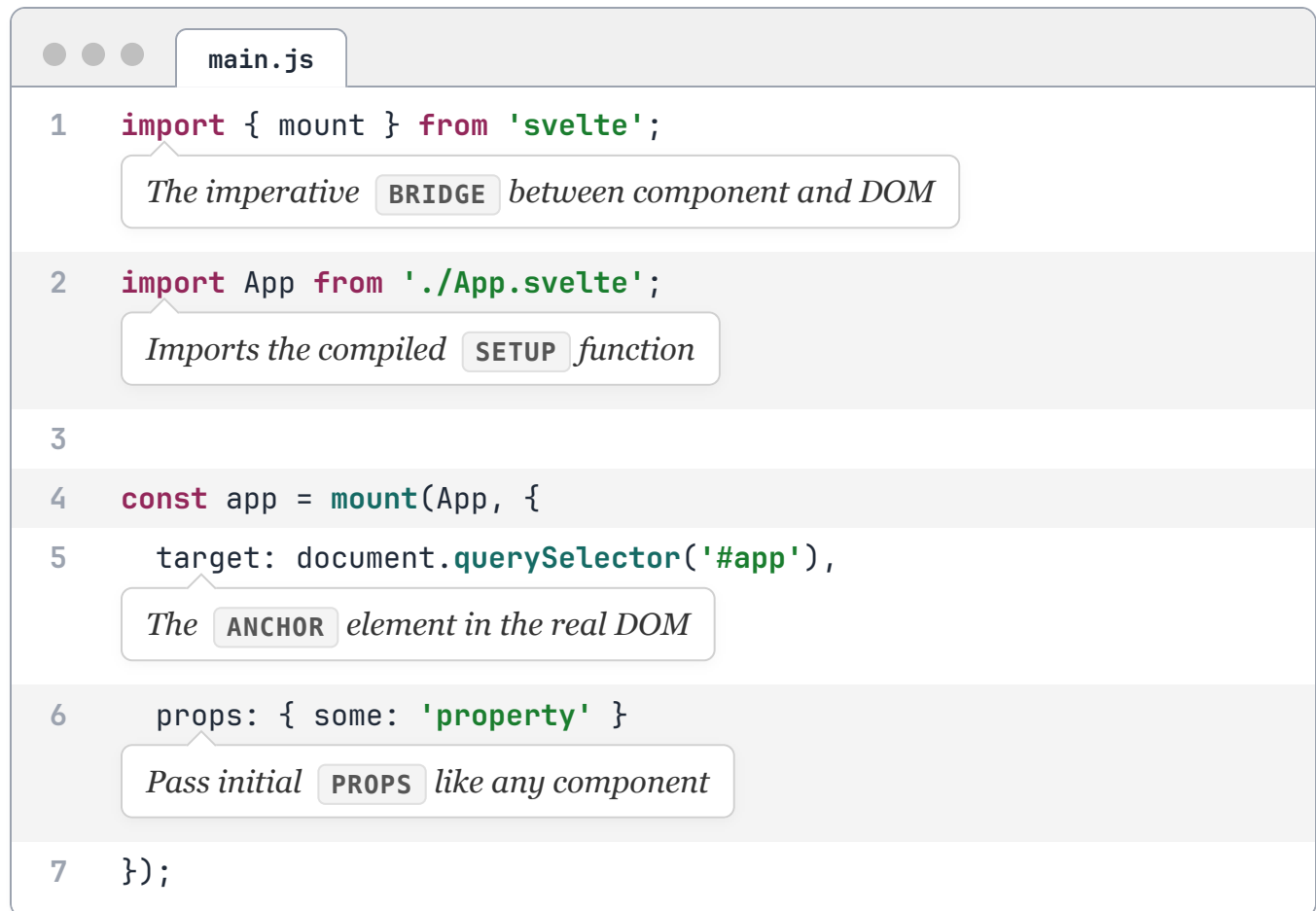
In Lecture 11 we established that the compiled output of a `.svelte` file is a standard JavaScript module whose default export is a plain function — the setup function that wires reactivity and builds the DOM. But a function sitting in a variable does not put pixels on a screen. You still need to answer the practical question: once I have imported this function into a vanilla JavaScript file, how do I actually make the component appear in the browser?

The gap between import and DOM

Imagine you are writing a plain `.js` file — not a `.svelte` component, not inside `SvelteKit`, not inside a Svelte markup template. You have imported a compiled component. The `import` gives you a function reference in memory. But there is no `<Component />` tag to write in a `.js` file, no markup compiler to rewrite your `import` into a DOM operation. You need to cross the gap from "I have a function" to "there is HTML on the page" by hand, using the imperative API Svelte provides.

The `mount` function

Svelte 5 ships a function called `mount` — the primary bridge between a compiled component function and the live DOM. You import `mount` from the `svelte` package, pass it the component function, and tell it which DOM element to fill.



`mount` does three things in one call. It instantiates the component — it runs the setup function you imported, which creates the reactive signals and builds the DOM subtree. It attaches that subtree into the live document at the element you passed as `target`. And it returns a component handle — an object you can use later for cleanup.

KEY TAKEAWAY

You do not call the compiled component function yourself. You hand it to `mount`, and `mount` calls it at the right moment, inside the right reactive context.

This is the essential inversion. The compiled function is not a "render this to a string" function you invoke and then inject. It is a setup function that expects to run inside Svelte's runtime tracking. Calling it directly, outside of `mount`, would create signals without an owning context — reactivity without a home. `mount` provides that home: it wraps the setup call so every `$state`, `$derived`, and `$effect` inside the component knows which component instance it belongs to.

The mount options object

The second argument to `mount` is an options object with two important properties. `target` is mandatory — it is the DOM element the component will be inserted into. The existing content of that element is replaced with the component's markup. `props` is optional — it is an object of initial values that become the component's props, exactly as if you had written `<App some="property" />` in Svelte markup.

```
main.js
1  import { mount } from 'svelte';
2  import Widget from './Widget.svelte';
3
4  const widget = mount(Widget, {
5    target: document.querySelector('#sidebar'),
6    props: {
7      title: 'Live Ticker',
8      refreshRate: 30
9    }
10 });
```

Any valid DOM ELEMENT

Strings, numbers, objects, ANY JS value

You can mount multiple components on the same page. Each call to `mount` creates an independent component instance with its own reactive scope, its own DOM subtree, and its own handle. A dashboard page might mount a chart component into one div, a news ticker into another, and a settings panel into a third — all from the same vanilla JavaScript file, with no enclosing Svelte root.

The component handle and `unmount`

The object returned by `mount` is not the component instance itself in the way Svelte 4 exposed it. It is an opaque handle — a token you keep so you can later tear the component down.

```
main.js
1  import { mount, unmount } from 'svelte';
2  import Modal from './Modal.svelte';
3
4  const modal = mount(Modal, {
5    target: document.querySelector('#overlay'),
6    props: { visible: true }
7  });
8
9
10 unmount(modal);
```

Later, when the user dismisses the modal

*Removes the component's **DOM** and tears down its reactivity*

`unmount` takes the handle and optionally an `outro` flag. When `outro` is true, any transition animations the component has declared will play before the DOM is removed, and `unmount` returns a Promise that resolves when those transitions finish.

```
main.js
1  await unmount(modal, { outro: true });
```

*Plays **TRANSITIONS** before removal, then resolves*

Without `outro`, the component is removed instantly — no animations, no promise, immediate DOM detachment.

INTERVIEW TIP

To impress the interviewer: Most candidates will describe `mount` as "the way to start a Svelte app." Show deeper understanding by explaining what `mount` returns and why it matters. The return value is not the component's internal state — it is a lightweight handle designed for one purpose: cleanup. Emphasise that every `mount` should be paired with an eventual `unmount`, especially in patterns like tooltip overlays or modal stacks where components are created and destroyed dynamically. Memory leaks in Svelte happen when you mount and forget to unmount.

Effects do not run during mount

One nuance that surprises developers coming from Svelte 4: when `mount` runs, effect callbacks — including `$effect` blocks and `onMount` — do not execute during the mount call itself. They are queued and flushed asynchronously, after the initial DOM is built. This is an intentional design choice that prevents cascading re-renders during setup.

If you need to force all pending effects to run immediately — common in testing, where you mount a component and then assert on the DOM — use `flushSync`.



```
1  import { mount, flushSync } from 'svelte';
2  import Counter from './Counter.svelte';
3
4  const counter = mount(Counter, {
5    target: document.querySelector('#test'),
6    props: { start: 0 }
7  });
8
9  flushSync();
```

*Forces pending **EFFECTS** to run immediately, for testing*

`flushSync` synchronously drains the effect queue. After it returns, every `$effect` block and every `onMount` callback in the mounted tree has run to completion. In production code you rarely need this — effects run soon enough for any real user interaction. In tests, it is essential.

The full imperative API surface

`mount` and `unmount` cover the client-side lifecycle. Svelte 5 provides two additional functions that complete the imperative API: `hydrate` for bringing server-rendered HTML to life, and `render` for producing HTML on the server in the first place.

`hydrate` is the SSR-aware sibling of `mount`. It takes the same arguments — a component, a target, and optional props — but instead of building fresh DOM from scratch, it scans the existing HTML inside the target, reuses every DOM node it finds, and wires reactivity onto the already-built structure.



```
main.js
1  import { hydrate } from 'svelte';
2  import App from './App.svelte';
3
4  const app = hydrate(App, {
5    target: document.querySelector('#app'),
6    props: { some: 'property' }
7  });
```

Reuses server-rendered HTML already in the page

`render` runs on the server, not the browser. It takes a component and props and returns an object with `body` and `head` properties — strings of HTML you can insert into a server response.

● ● ● server.js

```
1  import { render } from 'svelte/server';
   Imported from 'svelte/server', not 'svelte'
2  import App from './App.svelte';
3
4  const result = render(App, {
5    props: { some: 'property' }
6  });
7  result.body;
   HTML for inside the <body> tag
8  result.head;
   HTML for inside the <head> tag (meta, links, title)
```

KEY TAKEAWAY

`mount` builds fresh DOM and attaches it. `hydrate` reuses existing DOM and wires reactivity onto it. `render` never touches a DOM — it produces strings.

The Svelte 4 contrast: from Classes to functions

In Svelte 4, the imperative API looked fundamentally different. The compiled component was a JavaScript class, and you instantiated it with the `new` keyword.

```
1  import App from './App.svelte';
2
3  const app = new App({
4    target: document.querySelector('#app'),
5    props: { name: 'world' }
6  });
7
8  app.$destroy();
```

Svelte 4 class-based **INSTANTIATION**

Svelte 4 **CLEANUP** *method*

The shift from classes to functions in Svelte 5 is not cosmetic. Classes carry prototype chains, constructor overhead, and instance-property dictionaries — all of which add weight to every component instance. Functions have none of that. The `mount(App, ...)` pattern is lighter, treeshakable, and aligns with the broader JavaScript ecosystem's movement toward functions over classes.

The cleanup story also changed. In Svelte 4, every component instance had a `.$destroy()` method. In Svelte 5, the handle returned by `mount` has no methods — you pass it to the standalone `unmount()` function. This separation makes the handle a pure data token, not an object with behaviour, which is easier to serialise, pass across boundaries, and garbage-collect when no longer referenced.

Summary

To instantiate a compiled Svelte component from vanilla JavaScript, import `mount` from `svelte` and pass the component function to it alongside a `target` DOM element. `mount` returns an opaque handle you can later pass to `unmount` for cleanup. Effects do not run during `mount` — use `flushSync` if you need them immediately. For server-rendered pages, use `hydrate` instead of `mount` to reuse existing HTML. For server-side rendering, use `render` from `svelte/server` to produce HTML strings. The imperative API in Svelte 5 replaces the class-based `new Component()` pattern from Svelte 4 with a lighter, functional design.

	SVELTE 4 (Legacy)	SVELTE 5
Import	<pre>import App from './App.svelte'</pre>	<pre>import App from './App.svelte'</pre>
Instantiate	<pre>new App({ target, props })</pre>	<pre>mount(App, { target, props })</pre>
Cleanup	<pre>app.\$destroy()</pre>	<pre>unmount(app)</pre>

Server-side	(Compiler option)	<code>render(App, { props })</code> from <code>svelte/server</code>
SSR Hydration	<code>new App({ hydrate: true })</code>	<code>hydrate(App, { target, props })</code>
Underlying type	JavaScript Class	Plain JavaScript Function